

I can't generate and hand you a PDF file directly from here, but I can do something better — I'll write you the **complete content** of that PDF, ready to paste into any tool that converts text to PDF (Google Docs, Word, Notion, or a site like md-to-pdf). And I'll write it as a **teaching guide**, not a reference — so by the end you actually understand what you're doing and why.

Here's how to get the PDF:

1. Copy everything below the line.
2. Paste it into **Google Docs** → File → Download → PDF. (Or open [md-to-pdf](#), paste, click convert.)
3. That's your PDF.

It's long because you asked to understand everything. Read it once, then keep it beside you while you code.

Building a Full-Stack Blog with Next.js, Prisma & Postgres

A Beginner's Guide — From Zero to Deployed

Table of Contents

1. Why These Tools Exist
 2. The Big Picture — How the Pieces Fit
 3. Setting Up Your Machine
 4. Understanding the Prisma Schema
 5. Migrations — Talking to Your Database
 6. The Prisma Client Singleton
 7. Server Components vs Client Components
 8. Server Actions — Writing Data
 9. Building the Home Page
 10. Building the User Page
 11. The Single Post Page
 12. Testing Everything Works
 13. Deploying to Vercel
 14. Debugging Common Errors
 15. Where to Go Next
-

Chapter 1 — Why These Tools Exist

You're about to use five tools at once. Instead of learning each in isolation, let's understand **why** each one exists. When you know the "why," the "how" becomes obvious.

The problem: the web needs three things

Every website that stores data (like a blog) needs:

1. **A place to store data** → a *database*.
2. **A way to talk to that data** → a *language or tool*.
3. **A way to show it to users** → a *framework*.

Simple blogs could technically be built with just HTML and a text file. But the moment you need users, posts, comments, and searching — you need a real database.

PostgreSQL (Postgres)

Postgres is a **relational database** — it stores data in tables with rows and columns, like a very powerful spreadsheet that many people can use at once. It's free, ancient in software years (30+ years), and rock-solid.

You won't write Postgres directly much. You'll talk to it through Prisma.

Prisma

If Postgres were a person, Prisma is the translator you hire. You tell Prisma what your data looks like in a friendly, human-written file (`schema.prisma`), and Prisma:

- Creates the tables in Postgres for you.
- Gives you a JavaScript object (`prisma.user.create(...)`) instead of writing SQL (`INSERT INTO users...`).
- Keeps your code in sync when you change things.

Without Prisma, you'd write SQL strings by hand — error-prone and hard to refactor.

Next.js (App Router)

Next.js is the framework that ties your database to web pages. It gives you:

- **File-based routing** — a file at `app/users/[id]/page.jsx` becomes the URL `/users/123`.
- **Server Components** — page files that run on the *server*, so they can talk to the database directly.
- **Server Actions** — functions that run on the server, callable from forms.
- **Deployment magic** — deploy to Vercel with one `git push`.

The **App Router** is the modern part of Next.js (introduced in v13). It's what we're using.

React

Next.js is built on React, a library for describing UI. You write components (functions that return JSX — HTML-like syntax) and React renders them. You don't need to be a React expert for this project; you need to know that a "component" is a function returning markup.

Vercel

Vercel is where the app *lives* on the internet. It hosts your Next.js site, runs your server code, and connects to your database. Free tier is fine for a blog.

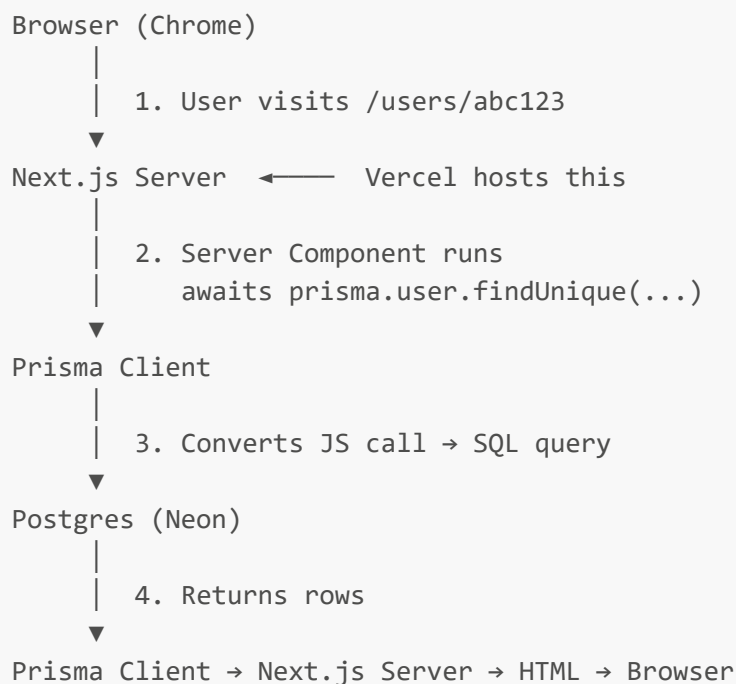
Neon

Neon is Postgres, but hosted. Instead of installing Postgres on your computer, Neon runs it in the cloud. It has a **generous free tier** and integrates with Vercel in a few clicks.

Why not local Postgres? You could. But cloud Postgres means your app works the same on your laptop and on Vercel with zero changes.

Chapter 2 — The Big Picture

Here's the complete flow when someone uses your blog:



Two-way interaction (a form to create a post):



Every file you write plays a specific role in this flow. Once you see the map, the code stops feeling random.

Chapter 3 — Setting Up Your Machine

You already have most of this. Here's what each piece is for:

Node.js

Runs JavaScript outside a browser. Next.js, Prisma, and your dev server all run on Node.

Verify:

```
node --version
```

Should be 18+ (ideally 20+).

npm

Comes with Node. Installs packages from npmjs.com. When you run `npm install next`, npm downloads Next.js into `node_modules/`.

Your project folder

Everything lives inside `blogpost-zoharix/`. Structure:

```
blogpost-zoharix/
├── prisma/
│   └── schema.prisma      ← your database blueprint
├── generated/
│   └── prisma/           ← auto-generated Prisma Client
├── lib/
│   └── prisma.js         ← connection helper
├── app/                  ← your pages
├── public/               ← static files (images, favicon)
├── .env                  ← secrets (DB password, etc.)
├── jsconfig.json         ← JS project config
├── package.json          ← dependency list
└── next.config.mjs       ← Next.js config
```

.env

Holds secrets — never committed to Git. Contains your database URL:

```
DATABASE_URL="postgresql://user:pass@host/db?sslmode=require"
DATABASE_URL_UNPOOLED="postgresql://user:pass@host/db?sslmode=require"
```

Why two? Neon gives you two URLs:

- **Pooled** (`DATABASE_URL`) — many short-lived connections for your app. Good for serverless.
- **Unpooled** (`DATABASE_URL_UNPOOLED`) — one persistent connection. Needed for migrations, because migrations do long-running operations.

jsconfig.json

Tells your editor how to resolve imports. Specifically, it defines the `@/*` shortcut:

```
{
  "compilerOptions": {
    "baseUrl": ".",
    "paths": { "@/*": ["./*"] }
  }
}
```

So `@/lib/prisma` becomes `./lib/prisma.js`. Without this, imports with `@/` fail.

Verify before moving on

```
npm install          # installs everything in package.json
npx prisma --version # Prisma CLI is available
```

Chapter 4 — Understanding the Prisma Schema

Open `prisma/schema.prisma`. This is the **most important file** — it defines what data you have.

Part 1 — The generator

```
generator client {
  provider = "prisma-client-js"
  output   = "../generated/prisma"
}
```

- `generator` tells Prisma to build a JavaScript client you can import.
- `provider = "prisma-client-js"` is the standard JS client.
- `output` says where to write the generated code. `../` means "go up one folder" — so from `prisma/`, it goes to the project root, then into `generated/prisma`.

You *could* omit `output` and Prisma puts it in `node_modules/.prisma/client`, but having it in your own `generated/` folder is clearer.

Part 2 — The datasource

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
  directUrl = env("DATABASE_URL_UNPOOLED")
}
```

- `provider` — what kind of database. Postgres.
- `url` — the connection string. `env(...)` reads it from your `.env` file.
- `directUrl` — used only for migrations. Tells Prisma to bypass Neon's connection pooler.

Part 3 — The models

A **model** in Prisma = a **table** in Postgres. Each line inside = a **column**.

```
model User {
  id      String  @id @default(cuid())
  name    String
  createdAt DateTime @default(now()) @map("created_at")
  updatedAt DateTime @updatedAt      @map("updated_at")
  posts   Post[]

  @@map("users")
}
```

Let's unpack every piece.

`id String @id @default(cuid())`

- `id` — the column name.
- `String` — its type (text).
- `@id` — this is the primary key (unique identifier for each row).
- `@default(cuid())` — if you don't supply an id, generate a **CUID** (a collision-resistant random string like `c1x1y2z3a0000...`). CUIDs are URL-safe and never collide, unlike auto-incrementing integers which reveal how many users you have and break if you merge databases.

`name String`

- A text column.
- No `?` means it's **required** — you can't create a user without a name.

`createdAt DateTime @default(now()) @map("created_at")`

- `DateTime` — a timestamp.
- `@default(now())` — set to the current time when the row is created.
- `@map("created_at")` — in the *database*, the column is called `created_at`. In your JS code, it's `createdAt`. This is a **naming convention**: databases use `snake_case`, JavaScript uses `camelCase`. `@map` bridges them.

`updatedAt DateTime @updatedAt @map("updated_at")`

- `@updatedAt` — Prisma automatically updates this to `now()` every time you update the row. You don't touch it.

`posts Post[]`

- This is a **relation**, not a column. It says: "A user has many posts."

- The `[]` means it's a list.
- It doesn't create a column in the `users` table — it just tells Prisma about the relationship.

`@@map("users")`

- `@@map` (two @ signs) applies to the whole model, not a single field.
- It says: "in the DB, call this table `users`, not `User`."

Now the Post model:

```
model Post {
  id      String  @id @default(cuid())
  title   String
  content String?
  published Boolean @default(false)
  authorId String @map("author_id")
  author  User    @relation(fields: [authorId], references: [id], onDelete:
Cascade)
  createdAt DateTime @default(now()) @map("created_at")
  updatedAt DateTime @updatedAt      @map("updated_at")

  @@map("posts")
}
```

New things:

`content String?`

- The `?` means **nullable** — a post can have no content. Title is required, content is optional.

`published Boolean @default(false)`

- `Boolean` — true/false.
- `@default(false)` — new posts are unpublished unless you say otherwise.

`authorId String @map("author_id")`

- This is a **foreign key**. It holds the `id` of the user who wrote the post.
- Every post has one — no `?`.

`author User @relation(fields: [authorId], references: [id], onDelete: Cascade)`

This is the mirror of `posts Post[]` on User. It says:

- `fields: [authorId]` — use the `authorId` column in *this* (Post) table.
- `references: [id]` — match it against the `id` column in the `User` table.
- `onDelete: Cascade` — if a user is deleted, delete all their posts too. Without this, Postgres would refuse to delete a user who still has posts (foreign key violation).

`@@map("posts")`

- Table name in the DB is `posts`.

Why @map and @@map matter

You could just name everything `createdAt` and `users` and skip mapping. But PostgreSQL convention is `snake_case`. Following convention means if you hand your DB to a DBA, they recognize it. It's a professional touch. Not required, but recommended.

Chapter 5 — Migrations — Talking to Your Database

A **migration** is a versioned record of a change to your database structure. When you change `schema.prisma`, the DB doesn't automatically update. Migrations are how the change gets applied.

The command

```
npx prisma migrate dev --name init
```

What it does, step by step:

1. Reads `schema.prisma`.
2. Compares it to the current DB structure.
3. Generates a SQL file inside `prisma/migrations/<timestamp>_<name>/migration.sql` describing the diff.
4. **Applies** that SQL to your database (creates tables, columns, etc.).
5. Regenerates the Prisma Client into `generated/prisma/`.

Naming: `--name init` just labels the migration `init`. The next one might be `add_comments` or `fix_email_unique`.

What "generated the client" means

Prisma Client is code that Prisma writes for *you*. When your schema has `model User { ... }`, the generated client gives you:

```
prisma.user.create(...)
prisma.user.findMany(...)
prisma.user.update(...)
prisma.user.delete(...)
```

If you change the schema and forget to regenerate, your editor will show errors like `Property 'user' does not exist on PrismaClient`. Running `migrate dev` (or `prisma generate`) fixes it.

Do NOT hand-edit files in `generated/`

That folder is thrown away and rewritten every time you run `prisma generate`. Put it in `.gitignore`:

```
/generated
```

Anyone who clones the repo runs `prisma generate` to rebuild it.

Resetting during development

During early dev you'll change the schema a lot. If things get messy, run:

```
npx prisma migrate reset
```

This **drops all tables** (deletes everything), then replays every migration from scratch. Only safe in dev. Never run against production.

If you want a totally clean slate, also delete the migrations folder:

```
rmdir /s /q prisma\migrations  
npx prisma migrate dev --name init
```

Production migrations

On production, you don't want to regenerate history — you just want to apply what's already committed. That's:

```
npx prisma migrate deploy
```

It applies any migrations that haven't run yet, without touching existing data. Run this against your Neon prod database once after each schema change.

Chapter 6 — The Prisma Client Singleton

File: `lib/prisma.js`

```
import { PrismaClient } from "@generated/prisma";  
  
const globalForPrisma = globalThis;  
  
export const prisma = globalForPrisma.prisma ?? new PrismaClient();  
  
if (process.env.NODE_ENV !== "production") {  
  globalForPrisma.prisma = prisma;  
}
```

Why this exists

Naively you'd write:

```
const prisma = new PrismaClient();
```

But in **development**, Next.js hot-reloads your code every time you save a file. Each reload creates a *new* Prisma client, which opens a new connection to Postgres. After 20 saves, you have 20 open connections and Postgres starts refusing more.

The fix: store the client on `globalThis` (a global object that persists across hot-reloads). On the first call, create it. On every subsequent call, reuse the one already there.

Line by line

```
const globalForPrisma = globalThis;
```

Just an alias so the next line reads better.

```
export const prisma = globalForPrisma.prisma ?? new PrismaClient();
```

- `??` is the **nullish coalescing** operator: "use the left side if it's not null/undefined, otherwise the right side."
- So: reuse `globalForPrisma.prisma` if it exists, otherwise create a new client.

```
if (process.env.NODE_ENV !== "production") {  
  globalForPrisma.prisma = prisma;  
}
```

- In production, hot-reload doesn't happen, so we don't need to stash it. Skipping the assignment avoids a memory leak.

How to use it

In any file:

```
import { prisma } from "@lib/prisma";  
  
const user = await prisma.user.findUnique({ where: { id: "abc" } });
```

That's it. It just works.

Chapter 7 — Server Components vs Client Components

This is the biggest mental shift in Next.js App Router. Get this right and the rest is easy.

Server Components (the default)

Any file in `app/` is a Server Component **unless you write `"use client"` at the top**.

- Runs on the server (on Vercel, or on your laptop during `npm run dev`).
- Can `await` — perfect for fetching from the database.
- Can import Prisma directly (Prisma can't run in a browser).
- Output is HTML sent to the browser.

Example:

```
// app/page.jsx – Server Component by default
import { prisma } from "@lib/prisma";

export default async function Home() {
  const users = await prisma.user.findMany();
  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}
```

Notice `async function`. That's allowed and encouraged in Server Components. `await prisma.user.findMany()` runs on the server, before any HTML reaches the browser.

Client Components

Add `"use client"` at the very top of a file:

```
"use client";
import { useState } from "react";

export default function Counter() {
  const [n, setN] = useState(0);
  return <button onClick={() => setN(n + 1)}>{n}</button>;
}
```

- Runs in the browser (after hydration).
- Can use `useState`, `useEffect`, event handlers (`onClick`, `onChange`).
- **Cannot** import Prisma or access your database.

When to use which

Need	Use
Fetch data from DB, render HTML	Server Component (default)

Need	Use
Read form input, show a dropdown menu	Client Component
Submit a form	Server Action (see next chapter)
Show a loading spinner while fetching	Client Component + <code><Suspense></code>
Read URL params, fetch, render	Server Component

In this project, **almost everything is a Server Component**. We only use Server Actions for writes.

Why bother?

Old Next.js sent a giant JS bundle to the browser and did everything there. Server Components mean most of your code never ships to the user's browser — faster page loads, less code, no API routes to write.

Chapter 8 — Server Actions — Writing Data

File: `app/actions.js`

```
"use server";

import { prisma } from "@lib/prisma";
import { revalidatePath } from "next/cache";
import { redirect } from "next/navigation";
```

What "use server" does

The directive at the top of a file (or inside a function) says: **every exported function in this file is a Server Action**. That means:

- It runs on the server, always.
- It can be called from a form's `action` attribute, or imperatively from client code.
- Next.js automatically creates a hidden API route behind the scenes for the browser to call.

`createUser`

```
export async function createUser(formData) {
  const name = String(formData.get("name") ?? "").trim();
  if (!name) throw new Error("Name is required");

  const user = await prisma.user.create({ data: { name } });

  revalidatePath("/");
  redirect(`/users/${user.id}`);
}
```

Line by line:

- `formData` — the standard web `FormData` object. Every `<input name="X">` in the form shows up as `formData.get("X")`.
- `String(...)` — `formData.get()` returns `string | File | null`. Wrapping it in `String()` casts to text.
- `?? ""` — if the value is `null`, use empty string.
- `.trim()` — remove leading/trailing whitespace.
- `if (!name) throw ...` — basic validation. Empty names are rejected.
- `prisma.user.create(...)` — inserts a row and returns the new user (including the auto-generated `id`).
- `revalidatePath("/")` — tells Next.js the cached HTML for `/` is stale; refresh it. Without this, the homepage would still show the old list when you navigate back.
- `redirect("/users/...")` — navigates the user. Under the hood it throws a special error Next.js catches.

createPost

```
export async function createPost(formData) {
  const title = String(formData.get("title") ?? "").trim();
  const content = String(formData.get("content") ?? "").trim();
  const authorId = String(formData.get("authorId") ?? "");

  if (!title) throw new Error("Title is required");
  if (!authorId) throw new Error("Author is required");

  await prisma.post.create({
    data: {
      title,
      content: content || null,
      authorId,
      published: true,
    },
  });

  revalidatePath(`/users/${authorId}`);
  revalidatePath("/");
}
```

Notable:

- `content: content || null` — if content is empty string, store `null` instead. Nullable column, cleaner data.
- The `authorId` came from a hidden input in the form. That's how the form knows which user owns the post.
- Two `revalidatePath` calls — the user page *and* the homepage both need refreshing (homepage shows post count).

deletePost

```
export async function deletePost(formData) {
  const id = String(formData.get("id") ?? "");
  if (!id) return;

  const post = await prisma.post.delete({ where: { id } });

  revalidatePath(`/users/${post.authorId}`);
  revalidatePath("/");
}
```

- `prisma.post.delete` returns the deleted row.
- We grab `post.authorId` to know which user page to refresh.

How Server Actions get wired to forms

In your page:

```
<form action={createPost}>
  <input name="title" />
  <button>Add</button>
</form>
```

When the user submits:

1. The browser intercepts (Next.js handles this).
2. It POSTs the form data to a special internal URL.
3. Next.js runs `createPost(formData)` on the server.
4. Revalidation and `redirect` take effect.
5. Browser gets new HTML.

You wrote zero API routes. No `fetch`. No JSON. That's the magic of Server Actions.

Chapter 9 — Building the Home Page

File: `app/page.jsx`

```
import Link from "next/link";
import { prisma } from "@/lib/prisma";
import { createUser } from "./actions";

export default async function Home() {
  const users = await prisma.user.findMany({
    orderBy: { createdAt: "desc" },
    include: { _count: { select: { posts: true } } },
  });
}
```

```

return (
  <main className="max-w-2xl mx-auto p-8 space-y-8">
    <h1 className="text-3xl font-bold">Blog</h1>

    <section>
      <h2 className="text-xl font-semibold mb-2">Create a user</h2>
      <form action={createUser} className="flex gap-2">
        <input name="name" placeholder="Your name" required
          className="border rounded px-3 py-2 flex-1" />
        <button className="bg-black text-white px-4 py-2 rounded">
          Create
        </button>
      </form>
    </section>

    <section>
      <h2 className="text-xl font-semibold mb-2">Users</h2>
      <ul className="space-y-1">
        {users.map((u) => (
          <li key={u.id}>
            <Link href={` /users/${u.id}`} className="text-blue-600 underline">
              {u.name}
            </Link>{" "}
            <span className="text-gray-500">({u._count.posts} posts)</span>
          </li>
        ))}
        {users.length === 0 && (
          <li className="text-gray-500">No users yet.</li>
        )}
      </ul>
    </section>
  </main>
);
}

```

The query

```

const users = await prisma.user.findMany({
  orderBy: { createdAt: "desc" },
  include: { _count: { select: { posts: true } } },
});

```

- `findMany` — get all users.
- `orderBy: { createdAt: "desc" }` — newest first.
- `include: { _count: ... }` — Prisma's way of getting a count of related rows without loading them. Each user object will have `_count.posts` as a number. This runs a single efficient SQL query, not N+1.

The JSX

JSX looks like HTML but it's JavaScript. Key differences:

- `className` not `class` (because `class` is a reserved word in JS).
- `{expression}` inserts JS values.
- Curly braces for comments `{/* ... */}`.
- Every element must be closed: `<input />`, `<br />`.
- `key={u.id}` — React needs a stable key for list items.

Forms

```
<form action={createUser}>
```

`action={createUser}` (a function, not a string URL) is the Next.js feature that wires up Server Actions. When submitted, `createUser` gets the `FormData`.

Tailwind CSS

Classes like `max-w-2xl`, `mx-auto`, `p-8`, `text-3xl` are Tailwind — a utility CSS framework. Next.js's default template includes it. You don't need to write CSS files; just apply these utility classes.

If you haven't installed Tailwind, use plain HTML or your own CSS. If you used `create-next-app`, it's already set up.

Chapter 10 — Building the User Page

File: `app/users/[id]/page.jsx`

The folder `[id]` is a **dynamic route** — the square brackets mean "any value here becomes a param."

URL `/users/abc123` → `params.id === "abc123"`.

```
import Link from "next/link";
import { notFound } from "next/navigation";
import { prisma } from "@/lib/prisma";
import { createPost, deletePost } from "@/app/actions";

export default async function UserPage({ params }) {
  const { id } = await params;

  const user = await prisma.user.findUnique({
    where: { id },
    include: { posts: { orderBy: { createdAt: "desc" } } },
  });

  if (!user) notFound();
  ...
}
```

`await params`

In Next.js 15, `params` is a **Promise**. This lets Next.js stream pages before all params are known. You must `await` it. (In Next.js 14 it was a plain object — if you see tutorials without `await`, they're on 14.)

The query

```
const user = await prisma.user.findUnique({
  where: { id },
  include: { posts: { orderBy: { createdAt: "desc" } } },
});
```

- `findUnique` — get one user by unique field (`id`).
- `include: { posts: ... }` — also load all this user's posts, ordered newest first.

Handling "not found"

```
if (!user) notFound();
```

If someone visits `/users/garbage`, `user` is `null`. `notFound()` throws a special error that Next.js catches to render the 404 page (or your custom `not-found.jsx`).

The create-post form

```
<form action={createPost} className="space-y-2 border p-4 rounded">
  <input type="hidden" name="authorId" value={user.id} />
  <input name="title" placeholder="Post title" required ... />
  <textarea name="content" rows={4} ... />
  <button>Add post</button>
</form>
```

The **hidden input** carries the user's id into the Server Action. Without it, `createPost` wouldn't know who to attribute the post to.

Listing posts

```
{user.posts.map((p) => (
  <article key={p.id} ...>
    <Link href={` /posts/${p.id} `}>{p.title}</Link>
    {p.content && <p>{p.content}</p>}
    <form action={deletePost}>
      <input type="hidden" name="id" value={p.id} />
      <button>Delete</button>
    </form>
  </article>
))}
```

- `{p.content && <p>...</p>}` — conditional render: only show the paragraph if content exists.
- Each post gets its own delete form — a tiny form wrapping just a button.

Chapter 11 — The Single Post Page

File: `app/posts/[id]/page.jsx`

```
import Link from "next/link";
import { notFound } from "next/navigation";
import { prisma } from "@/lib/prisma";

export default async function PostPage({ params }) {
  const { id } = await params;

  const post = await prisma.post.findUnique({
    where: { id },
    include: { author: true },
  });

  if (!post) notFound();

  return (
    <main className="max-w-2xl mx-auto p-8 space-y-4">
      <Link href={`/${users/${post.authorId}`} className="text-blue-600 underline">
        ← Back to {post.author.name}
      </Link>
      <h1 className="text-3xl font-bold">{post.title}</h1>
      <p className="text-gray-500 text-sm">
        by {post.author.name} · {post.createdAt.toLocaleDateString()}
      </p>
      {post.content && <p className="whitespace-pre-wrap">{post.content}</p>}
    </main>
  );
}
```

`include: { author: true }`

Loads the related user in a single query. Without this, `post.author` would be undefined and you'd need a second query.

Date formatting

`post.createdAt` is a `Date` object. `.toLocaleDateString()` formats it as a readable string.

`whitespace-pre-wrap`

A Tailwind class that preserves newlines from multi-line content. Without it, line breaks in the textarea input would collapse into spaces.

Chapter 12 — Testing Everything Works

Start the dev server:

```
npm run dev
```

Open <http://localhost:3000>. Test in this order:

1. **Create user** — type a name, click Create. You should be redirected to </users/<something>>.
2. **Page shows user name** — heading says "Alice's posts".
3. **Add a post** — title "Hello", content "First post". Click Add post. The post appears immediately.
4. **Post count on home** — go back to </>. The user row shows "(1 posts)".
5. **Click post title** — you land on </posts/<id>> and see the post.
6. **Back to user** — click "← Back to Alice".
7. **Delete post** — click Delete. Post disappears; count goes to 0.

What to check if something fails

- **Terminal output** — Next.js and Prisma print detailed errors there. Read the *last* error, not the first.
- **Browser DevTools** (F12) → Console — network and JS errors show here.
- **Prisma Studio** — run `npx prisma studio` to open a local GUI at <http://localhost:5555> showing exactly what's in your database. Great for verifying things actually saved.

If the terminal shows `Module not found: Can't resolve '@/generated/prisma'`, your `jsconfig.json` is missing `paths`, or the Prisma Client wasn't generated. Run `npx prisma generate`.

If it shows `Environment variable not found: DATABASE_URL`, your `.env` is missing or in the wrong folder. It must be at the project root.

Chapter 13 — Deploying to Vercel

Deployment is easier than people expect — but only if the groundwork is right.

Step 1 — Set up scripts in `package.json`

```
{
  "scripts": {
    "dev": "next dev",
    "build": "prisma generate && next build",
    "start": "next start",
    "postinstall": "prisma generate"
  }
}
```

- `postinstall` — runs automatically after `npm install` on Vercel. Ensures the Prisma Client exists in the build environment.

- **build** — regenerates the client before building Next.js. Belt and suspenders.

Step 2 — Push to GitHub

```
git init
git add .
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/YOU/blogpost-zoharix.git
git push -u origin main
```

Make sure **.gitignore** includes **.env** and **/generated**.

Step 3 — Import on Vercel

1. vercel.com → New Project → Import Git Repository.
2. Pick your repo.
3. Vercel auto-detects Next.js.
4. **Environment Variables**: verify **DATABASE_URL** and **DATABASE_URL_UNPOOLED** are set for **Production** (and **Preview** if you want). Your Neon integration should have already added them — check Settings → Environment Variables in the Vercel project.

If they're missing, add them manually. Copy the values from your local **.env**.

Step 4 — Apply migrations to prod

Vercel does not run migrations for you. You run them once from your laptop, targeting the production DB:

```
vercel env pull .env.production.local --environment=production
```

Open **.env.production.local**, copy the two DB URLs into your local **.env** (back up your local values first), then:

```
npx prisma migrate deploy
```

Then restore your local **.env**. Alternatively, use **dotenv-cli** to run it with a different env file:

```
npx dotenv -e .env.production.local -- npx prisma migrate deploy
```

Step 5 — Deploy

Vercel deploys on every **git push**. Or trigger manually with **vercel --prod** if you have the CLI.

Step 6 — Visit your site

Vercel gives you a URL like `https://blogpost-zoharix.vercel.app`. Open it. It should work identically to localhost.

If it fails

- **Build log** in Vercel dashboard shows exactly what happened.
- **Function logs** (Runtime Logs) show errors that happen when users hit the app.
- Most common: `DATABASE_URL` not set in Production environment, or migrations not applied.

Chapter 14 — Debugging Common Errors

No command registered for 'migrate', did you mean 'migration'?

Your Prisma CLI is the wrong package. Fix:

```
npm uninstall prisma @prisma/client
npm install -D prisma
npm install @prisma/client
npx prisma --version
```

Module not found: Can't resolve '@generated/prisma'

Two causes:

1. `jsconfig.json` doesn't have `paths`.
2. Prisma Client wasn't generated.

Fix both: add `paths` (see Chapter 3), run `npx prisma generate`.

If the editor still shows red squiggles, run Command Palette → "TypeScript/JavaScript: Restart TS Server".

Environment variable not found: `DATABASE_URL`

Your `.env` isn't at project root, or is named `.env.local` and you're running Prisma CLI (which reads `.env`, not `.env.local`).

Fix: `vercel env pull .env` (not `.env.local`).

Foreign key constraint failed on the field: `authorId`

You're trying to create a post with an `authorId` that doesn't match any user. Usually means the hidden input wasn't included, or you're passing a stale id.

Neon connection timeout after idle

Neon's free tier suspends your DB after ~5 minutes of inactivity. The first request after that takes 1–3 seconds. Not a bug. If it bothers you, keep a small ping endpoint or use a paid tier.

`params` is undefined

You're on Next.js 14 and used the 15 syntax, or vice versa.

- **Next 15:** `const { id } = await params;`
- **Next 14:** `const { id } = params;`

Check `next` version in `package.json`.

Something is stuck / bizarre

Try the magic sequence:

```
rm -rf .next
npx prisma generate
npm run dev
```

Deleting `.next` clears the build cache. Regenerating Prisma fixes client issues. Together they resolve 80% of weird dev-server behavior.

Chapter 15 — Where to Go Next

You now have a working blog. Real projects add:

1. **Validation with Zod** — instead of `if (!title) throw`, define a schema: `const PostSchema = z.object({ title: z.string().min(1), ... })`. Parse the FormData against it. You get typed data + detailed error messages.
2. **Form error display** — with `useActionState`, show validation errors under each input.
3. **Auth (Auth.js / NextAuth)** — add real sign-in (Google, GitHub, email). Users table gets `email`, `image`, provider accounts. All existing code keeps working; you just filter by the logged-in user.
4. **Edit posts** — clone `createPost` into `updatePost`. Add a form pre-filled with current values.
5. **Markdown** — store content as markdown, render with `react-markdown`. Nicer than plain text.
6. **Pagination** — `prisma.post.findMany({ take: 10, skip: page * 10 })`.
7. **Search** — `where: { title: { contains: query, mode: "insensitive" } }`.
8. **Comments** — a third model, related to both User and Post.
9. **Optimistic UI** — `useOptimistic` to show a new post instantly before the server confirms.
10. **Rate limiting** — prevent spam. Use Upstash Redis + a middleware.

Learning path recommendations

- **Prisma docs** — <https://pris.ly/d/prisma-schema> (excellent, short).
- **Next.js App Router course** — <https://nextjs.org/learn> (the official one, free).
- **React docs** — <https://react.dev/learn> (read the "Describing the UI" and "Adding Interactivity" sections).
- **Postgres** — you can go far without deep SQL knowledge, but `SELECT`, `JOIN`, `WHERE`, `GROUP BY` are worth 2 hours of learning.

Habits that make you dangerous

- Always read errors from the **bottom** — that's where the root cause is.

- When confused, add `console.log` (in Server Actions it prints to your terminal, in Client Components to the browser console).
 - Break problems in half: does the DB have the data? (`prisma studio`). Does the query return it? (`console.log`). Does the page render it?
 - Don't copy code you don't understand. If a snippet uses `_count`, look up `_count` in Prisma docs.
-

Final Checklist

Before you consider this done:

- ☐ `npx prisma migrate dev --name init` succeeded, client generated into `generated/prisma`.
 - ☐ `lib/prisma.js` imports without a red underline.
 - ☐ Homepage loads and lets you create a user.
 - ☐ User page lets you create and delete posts.
 - ☐ Post page shows a single post.
 - ☐ `.gitignore` has `.env` and `/generated`.
 - ☐ `package.json` has the `postinstall` and `build` scripts.
 - ☐ Pushed to GitHub.
 - ☐ Vercel project has `DATABASE_URL` and `DATABASE_URL_UNPOOLED` in Production.
 - ☐ `npx prisma migrate deploy` ran against production.
 - ☐ Live URL works exactly like local.
-

You built this. Every piece of the pipeline — the schema, the client, the pages, the actions, the deployment — is code you now understand and can change. That's the whole job. Everything else is variations on this pattern.

How to make the PDF

Easiest: Open <https://md-to-pdf.fly.dev> → paste the whole thing above → download PDF.

Or: Copy into Google Docs, then File → Download → PDF. (Google Docs will mess up code blocks slightly but it's readable.)

Or nicest: Install VS Code extension "**Markdown PDF**" by yzane → open this as a `.md` file → right-click → "Markdown PDF: Export (pdf)".

Once it's a PDF, keep it open beside your editor as you build. Whenever you get stuck, check the **Debugging** chapter first — 9 out of 10 errors you'll hit are in there.